

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0612	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Comparative Analysis
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	JAYAKRISHNA.V	Reg. No.:	192421259
Section / Batch:	1	Date:	27.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Coding Challenge Task	Comparative Analysis of Dynamic Programming and greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I JAYAKRISHNA.V(192421259)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: JAYAKRISHNA.V_____

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Knapsack Problem Comparison (0/1 DP vs Fractional Greedy)

1. Problem Understanding

The Knapsack Problem is a classical optimization problem in which a limited-capacity container must be filled with items to obtain the maximum possible value.

Each item has:

- A weight.
- A value.

The objective is to select items such that:

- The total weight does not exceed the capacity of the knapsack.
- The total value is maximized.

There are two important versions of the problem:

1. 0/1 Knapsack Problem
2. Fractional Knapsack Problem

In the 0/1 Knapsack Problem, an item must either be selected completely or not selected at all.

In the Fractional Knapsack Problem, an item can be divided, and a fraction of an item can be selected.

For the given problem:

Weights:

10, 20, 30

Values:

60, 100, 120

Knapsack Capacity:

50

The objective is to solve the problem using both Dynamic Programming for the 0/1 Knapsack Problem and the Greedy approach for the Fractional Knapsack Problem.

Mathematical Formulation

Let:

n = number of items

W = capacity of the knapsack

w_i = weight of item i

v_i = value of item i

0/1 Knapsack

For every item:

$x_i \in \{0, 1\}$

where:

$x_i = 1$ means the item is selected.

$x_i = 0$ means the item is not selected.

The objective is:

Maximize:

$$\sum v_i x_i$$

Subject to:

$$\sum w_i x_i \leq W$$

Fractional Knapsack

For the fractional version:

$$0 \leq x_i \leq 1$$

An item may be selected completely or partially.

The objective remains:

Maximize:

$$\sum v_i x_i$$

Subject to:

$$\sum w_i x_i \leq W$$

Input and Output Specification

Input

The program receives:

Weights = [10, 20, 30]

Values = [60, 100, 120]

Capacity = 50

Output

The program calculates:

- Maximum value using the 0/1 Knapsack algorithm.
- Maximum value using the Fractional Knapsack algorithm.
- Comparison of both approaches.

Expected results:

0/1 Knapsack Maximum Value: 220

Fractional Knapsack Maximum Value: 240.0

2. Constraint Analysis

Constraint analysis is important because the total weight of selected items must not exceed the capacity of the knapsack.

The following constraints are considered.

Capacity Constraint

The total weight of selected items must not exceed the available capacity.

Condition:

$$\text{Total Weight} \leq \text{Capacity}$$

For the given problem:

$$\text{Total Weight} \leq 50$$

This prevents selecting items whose combined weight exceeds the knapsack capacity.

Item Selection Constraint in 0/1 Knapsack

Each item can only be selected completely or rejected.

Condition:

$$x_i \in \{0, 1\}$$

For example:

$$\text{Item selected} \rightarrow x_i = 1$$

$$\text{Item not selected} \rightarrow x_i = 0$$

Partial selection is not allowed.

Fractional Selection Constraint

In the Fractional Knapsack Problem, an item can be partially selected.

Condition:

$$0 \leq x_i \leq 1$$

For example:

$$x_i = 0.5$$

means that 50% of the item is selected.

Value Maximization Constraint

The algorithm must select items that provide the highest possible total value while satisfying the capacity constraint.

The objective is:

Maximize Total Value

3. State-Space Representation and Selection Strategy

0/1 Knapsack Using Dynamic Programming

The state space can be represented using a Dynamic Programming table.

Let:

$$dp[i][w]$$

represent the maximum value that can be obtained using the first i items with a knapsack capacity of w .

For every item, there are two choices:

1. Do not select the item.
2. Select the item if sufficient capacity is available.

The recurrence relation is:

If:

$$w_i \leq w$$

then:

$$dp[i][w] = \max(dp[i-1][w], v_i + dp[i-1][w-w_i])$$

Otherwise:

$$dp[i][w] = dp[i-1][w]$$

Fractional Knapsack Using Greedy Method

The Fractional Knapsack Problem uses the value-to-weight ratio.

The ratio for each item is:

Value / Weight

For the given items:

Item 1:

$$60 / 10 = 6$$

Item 2:

$$100 / 20 = 5$$

Item 3:

$$120 / 30 = 4$$

The items are selected in descending order of their value-to-weight ratio.

Therefore:

Item 1 → Ratio 6

Item 2 → Ratio 5

Item 3 → Ratio 4

The greedy algorithm selects the items with the highest ratio first.

Selection Strategy

0/1 Knapsack

Possible decisions:

- Take Item 1 or leave Item 1.
- Take Item 2 or leave Item 2.
- Take Item 3 or leave Item 3.

The algorithm explores the possible combinations using Dynamic Programming.

Fractional Knapsack

The algorithm:

1. Sorts items by value-to-weight ratio.
 2. Selects the highest-ratio item.
 3. Continues while capacity is available.
 4. Takes a fraction of the next item if necessary.
-

4. Algorithm Using Dynamic Programming and Greedy Approach

Pseudocode for 0/1 Knapsack

FUNCTION zeroOneKnapsack(weights, values, capacity):

n = number of items

CREATE DP table of size $(n + 1) \times (\text{capacity} + 1)$

FOR i from 1 to n:

FOR w from 0 to capacity:

IF $\text{weights}[i - 1] \leq w$:

take = $\text{values}[i - 1] +$
 $\text{dp}[i - 1][w - \text{weights}[i - 1]]$

notTake = $\text{dp}[i - 1][w]$

$\text{dp}[i][w] = \text{maximum}(\text{take}, \text{notTake})$

ELSE:

$\text{dp}[i][w] = \text{dp}[i - 1][w]$

RETURN $\text{dp}[n][\text{capacity}]$

Pseudocode for Fractional Knapsack

FUNCTION fractionalKnapsack(weights, values, capacity):

CALCULATE value-to-weight ratio for every item

SORT items in descending order of ratio

totalValue = 0

FOR each item:

IF item weight $\leq$ remaining capacity:

take complete item

add item value to totalValue

ELSE:

take fraction of the item

add proportional value to totalValue

STOP

RETURN totalValue

5. Execution and Testing

Sample Input

Weights = [10, 20, 30]

Values = [60, 100, 120]

Capacity = 50

0/1 Knapsack Execution

The possible important combinations are:

Item 1 and Item 2

Weight:

$$10 + 20 = 30$$

Value:

$$60 + 100 = 160$$

This combination is valid.

Item 1 and Item 3

Weight:

$$10 + 30 = 40$$

Value:

$$60 + 120 = 180$$

This combination is valid.

Item 2 and Item 3

Weight:

$$20 + 30 = 50$$

Value:

$$100 + 120 = 220$$

This combination exactly matches the capacity.

Therefore:

Maximum Value = 220

Fractional Knapsack Execution

First, calculate the value-to-weight ratio.

Item 1

Value = 60

Weight = 10

Ratio = 6

Item 2

Value = 100

Weight = 20

Ratio = 5

Item 3

Value = 120

Weight = 30

Ratio = 4

The selection order is:

Item 1 → Item 2 → Item 3

Step 1

Select Item 1 completely.

Used Capacity:

10

Remaining Capacity:

$50 - 10 = 40$

Total Value:

60

Step 2

Select Item 2 completely.

Used Capacity:

20

Remaining Capacity:

$40 - 20 = 20$

Total Value:

$60 + 100 = 160$

Step 3

Item 3 has weight 30, but only 20 capacity remains.

Therefore, select:

$$20 / 30 = 2/3$$

of Item 3.

Value obtained:

$$(20 / 30) \times 120 = 80$$

Total Value:

$$160 + 80 = 240$$

Therefore:

Fractional Knapsack Maximum Value = 240.0

6. Correctness Justification

Correctness of 0/1 Knapsack Dynamic Programming

The Dynamic Programming algorithm considers both possible decisions for every item:

- Select the item.
- Do not select the item.

For each capacity, the algorithm stores the maximum possible value.

Therefore, every valid combination is considered indirectly through the DP table.

The recurrence selects the better value between:

Taking the current item.

and

Not taking the current item.

Thus, when the algorithm reaches:

$dp[n][capacity]$

it contains the optimal maximum value.

For the given input:

Maximum Value = 220

Therefore, the Dynamic Programming algorithm correctly solves the 0/1 Knapsack Problem.

Correctness of Fractional Greedy Algorithm

The Fractional Knapsack Problem allows items to be divided.

Therefore, selecting the item with the highest value-to-weight ratio first provides the greatest value for each unit of capacity.

The greedy algorithm:

1. Calculates the value-to-weight ratio.
2. Sorts items in descending order.
3. Selects the best ratio first.
4. Takes a fraction of an item when required.

Because fractional selection is allowed, this greedy strategy produces the optimal solution.

For the given input:

Maximum Value = 240.0

Therefore, the Fractional Greedy algorithm correctly solves the Fractional Knapsack Problem.

7. Complexity Analysis

0/1 Knapsack Using Dynamic Programming

Suppose:

n = number of items

W = knapsack capacity

The Dynamic Programming table contains:

$(n + 1) \times (W + 1)$

states.

Therefore, the time complexity is:

$O(nW)$

The space complexity is:

$O(nW)$

The space can be optimized in some implementations to:

$O(W)$

Fractional Knapsack Using Greedy Method

The main operation is sorting the items according to their value-to-weight ratio.

Sorting requires:

$O(n \log n)$

The selection process requires:

$O(n)$

Therefore, the overall time complexity is:

$O(n \log n)$

The additional space requirement depends on the implementation and is typically:

$O(n)$

for storing item information.

8. Comparison Between Both Approaches

Feature	0/1 Knapsack	Fractional Knapsack
Item Selection	Complete item only	Complete or partial item
Main Technique	Dynamic Programming	Greedy Algorithm
Greedy Always Optimal	No	Yes
Time Complexity	$O(nW)$	$O(n \log n)$
Memory Usage	Higher	Lower
Maximum Value	220	240.0

Feature	0/1 Knapsack	Fractional Knapsack
Item Division Allowed	No	Yes

Why Greedy Works Only for Fractional Knapsack

The greedy approach selects the item with the highest value-to-weight ratio.

For the Fractional Knapsack Problem, this is optimal because an item can be divided.

If there is not enough capacity for the entire item, the algorithm can take only the required fraction.

However, this is not possible in the 0/1 Knapsack Problem.

In the 0/1 version, taking an item completely may prevent the selection of a better combination of other items.

Therefore, a simple greedy strategy does not always produce the optimal solution for the 0/1 Knapsack Problem.

Dynamic Programming is used because it systematically evaluates the possible decisions.

9. Advantages and Limitations

Advantages of Dynamic Programming

- Produces the optimal solution for the 0/1 Knapsack Problem.
- Systematically evaluates possible item selections.
- Suitable when partial item selection is not allowed.
- Guarantees the maximum possible value.

Limitations of Dynamic Programming

- Requires more memory.
 - Time complexity depends on the capacity.
 - Can become expensive for very large capacities.
-

Advantages of Greedy Algorithm

- Simple to implement.
- Faster than Dynamic Programming for this problem.
- Requires less computational effort.
- Produces the optimal result for Fractional Knapsack.
- Suitable for resource allocation where partial allocation is allowed.

Limitations of Greedy Algorithm

- Does not always produce an optimal solution for 0/1 Knapsack.
 - Depends on the property of fractional selection.
 - Cannot handle indivisible items optimally in all cases.
-

Conclusion

The Knapsack Problem demonstrates the difference between Dynamic Programming and Greedy optimization techniques.

For the given items:

Weights = [10, 20, 30]

Values = [60, 100, 120]

Capacity = 50

The 0/1 Knapsack Problem produces:

Maximum Value = 220

The optimal selection is:

Item 2 and Item 3

The Fractional Knapsack Problem produces:

Maximum Value = 240.0

The selection includes:

Item 1 completely

Item 2 completely

2/3 of Item 3

The Fractional Knapsack achieves a higher value because items can be divided.

Thus, Dynamic Programming is appropriate for the 0/1 Knapsack Problem, while the Greedy approach is optimal and more efficient for the Fractional Knapsack Problem.

OUTPUT

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

JAYAKRISHNA V
192421259RC

CO4 AT3-COMPARATIVE ANALYSIS

← Back

Language: Python C C++ Java

QUESTIONS

Q1 Knapsack Problem Comparison (0/1 DP vs Fractional Greedy) 100 marks
100 marks

1/1 answered

Q1 Knapsack Problem Comparison (0/1 DP vs Fractional Greedy) 100 marks

Given items with weights [10, 20, 30] and values [60, 100, 120] with capacity 50, solve using both 0/1 knapsack (DP) and fractional knapsack (greedy). Compare the obtained values and selection strategy. Analyze why greedy provides optimal results only for fractional cases. Evaluate computational complexity. Discuss memory usage in DP. Generate insights on choosing appropriate approaches in resource allocation.

Your Code

```
1 weights = [10, 20, 30]
2 values = [60, 100, 120]
3 capacity = 50
4
5 n = len(weights)
6
7 dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]
8
9 for i in range(1, n + 1):
10     for w in range(capacity + 1):
11
12         if weights[i - 1] <= w:
13             take = values[i - 1] + dp[i - 1][w - weights[i - 1]]
14             not_take = dp[i - 1][w]
```

Input

stdin input

Output

0/1 Knapsack Maximum Value: 220
Fractional Knapsack Maximum Value: 240.0